

PAYMENT MANAGEMENT SYSTEM

PURPLLE ONLINE BEAUTY SHOPPING

1. Introduction

The **Purple Online Beauty Shopping Database Management System** is designed to efficiently manage customer payment information and payment transactions.

The system uses a **Payment** table related to the existing **Orders** table to store and organize payment details. Primary Key and Foreign Key constraints are used to maintain data integrity and establish a relationship between orders and payments.

The system stores payment mode, payment date, payment amount, and payment status. SQL operations are used to manage successful and failed transactions, analyze payment methods, and generate payment transaction reports.

The structure and sections below are adapted from the uploaded DBMS reference PDF, which uses **Payment_ID**, **Order_ID**, **Payment_Mode**, **Payment_Date**, **Payment_Amount**, and **Payment_Status** and includes successful/failed payments, status modification, payment-method analysis, and transaction reporting.

2. Objectives

- Create the **Payment** table.
- Define Primary Key and Foreign Key relationships.
- Store payment mode, date, amount, and status.
- Manage successful and failed payment transactions.
- Modify payment transaction status.
- Analyze payment methods used by customers.
- Calculate the amount collected through each payment mode.
- Generate payment transaction reports.
- Maintain data integrity using constraints.

3. Database Design

3.1 Payment Table

Attribute	Data Type	Key / Constraint	Description
Payment_ID	NUMBER	PK	Unique payment ID
Order_ID	NUMBER	FK	References Orders
Payment_Mode	VARCHAR2(20)	NOT NULL	Payment method
Payment_Date	DATE	NOT NULL	Date of payment
Payment_Amount	NUMBER(10,2)	NOT NULL	Amount paid
Payment_Status	VARCHAR2(20)	NOT NULL	Payment status

4. Table Relationships

Orders → Payment

Orders.Order_ID (PK) → Payment.Order_ID (FK)

Relationship: 1:1

ORDERS (1) — (1) PAYMENT

Customer → Orders

Customer5.Customer_ID (PK) → Orders.Customer_ID (FK)

Relationship: 1:M

CUSTOMER (1) — (M) ORDERS

Orders → Order_Details

Orders.Order_ID (PK) → Order_Details.Order_ID (FK)

Relationship: 1:M

ORDERS (1) — (M) ORDER_DETAILS

PurplleProduct → Order_Details

PurplleProduct.Product_ID (PK) → Order_Details.Product_ID (FK)

Relationship: 1:M

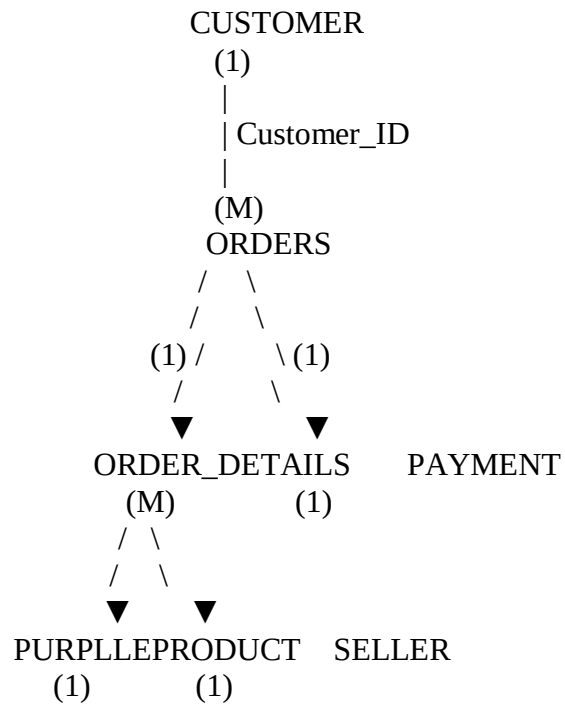
PURPLLEPRODUCT (1) — (M) ORDER_DETAILS

Seller → Order_Details

Seller.Seller_ID (PK) → Order_Details.Seller_ID (FK)

Relationship: 1:M

SELLER (1) — (M) ORDER_DETAILS



5. Table Creation

5.1 Create Payment Table

```

SQL> CREATE TABLE Payment (
2     Payment_ID NUMBER PRIMARY KEY,
3     Order_ID NUMBER NOT NULL,
4     Payment_Mode VARCHAR2(20) NOT NULL,
5     Payment_Date DATE NOT NULL,
6     Payment_Amount NUMBER(10,2) NOT NULL,
7     Payment_Status VARCHAR2(20) NOT NULL,
8     FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
9 );

```

Table created.

6. Payment Data Insertion

```

SQL> INSERT INTO Payment VALUES (701, 501, 'UPI', TO_DATE('20-09-2026','DD-MM-YYYY'), 1699.00, 'Successful');
1 row created.

SQL> INSERT INTO Payment VALUES (702, 502, 'CARD', TO_DATE('21-09-2026','DD-MM-YYYY'), 749.00, 'Successful');
1 row created.

SQL> INSERT INTO Payment VALUES (703, 503, 'COD', TO_DATE('22-09-2026','DD-MM-YYYY'), 1299.00, 'Successful');
1 row created.

SQL> INSERT INTO Payment VALUES (704, 504, 'UPI', TO_DATE('23-09-2026','DD-MM-YYYY'), 799.00, 'Failed');
1 row created.

SQL> INSERT INTO Payment VALUES (705, 505, 'CARD', TO_DATE('24-09-2026','DD-MM-YYYY'), 1599.00, 'Successful');
1 row created.

SQL> COMMIT;
Commit complete.

```

7. Data Retrieval

```
SQL> SELECT * FROM Payment;
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT
701	501	UPI	20-SEP-26	1699
702	502	CARD	21-SEP-26	749
703	503	COD	22-SEP-26	1299
704	504	UPI	23-SEP-26	799
705	505	CARD	24-SEP-26	1599

8. Manage Payment Transactions

8.1 Display Successful Payments

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Successful';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT
701	501	UPI	20-SEP-26	1699
702	502	CARD	21-SEP-26	749
703	503	COD	22-SEP-26	1299
705	505	CARD	24-SEP-26	1599

8.2 Display Failed Payments

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Failed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT
704	504	UPI	23-SEP-26	799

Failed

9. Modify Payment Status

```
SQL> UPDATE Payment SET Payment_Status = 'Successful' WHERE Payment_ID = 704;
```

1 row updated.

```
SQL> SELECT * FROM Payment WHERE Payment_ID = 704;
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT
704	504	UPI	23-SEP-26	799

Successful

```
SQL> COMMIT;
```

Commit complete.

10. Analyze Payment Methods

```
SQL> SELECT
2     Payment_Mode,
3     COUNT(*) AS Total_Transactions
4 FROM Payment
5 GROUP BY Payment_Mode;
```

PAYMENT_MODE	TOTAL_TRANSACTIONS
UPI	2
CARD	2
COD	1

11. Calculate amount collected by payment mode

```
SQL> SELECT
  2     Payment_Mode,
  3     SUM(Payment_Amount) AS Total_Amount
  4 FROM Payment
  5 WHERE Payment_Status = 'Successful'
  6 GROUP BY Payment_Mode;
```

PAYMENT_MODE	TOTAL_AMOUNT
UPI	2498
CARD	2348
COD	1299

12. Payment Transaction Report

```
SQL> SELECT
  2     P.Payment_ID,
  3     O.Order_ID,
  4     C.Customer_ID,
  5     C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  6     P.Payment_Mode,
  7     P.Payment_Date,
  8     P.Payment_Amount,
  9     P.Payment_Status
 10 FROM Payment P
 11 JOIN Orders O
 12   ON P.Order_ID = O.Order_ID
 13 JOIN Customers C
 14   ON O.Customer_ID = C.Customer_ID
 15 ORDER BY P.Payment_Date DESC;
```

PAYMENT_ID	ORDER_ID	CUSTOMER_ID

CUSTOMER_NAME		

PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS

705	505	105	
Sneha Patel			
CARD	24-SEP-26	1599	Successful

704	504	104	
Rahul Verma			
UPI	23-SEP-26	799	Successful

PAYMENT_ID	ORDER_ID	CUSTOMER_ID

CUSTOMER_NAME		

PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS

703	503	103	
Arun Kumar			
COD	22-SEP-26	1299	Successful

702	502	102	
Priya Sharma			

PAYMENT_ID	ORDER_ID	CUSTOMER_ID

CUSTOMER_NAME		

PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS

CARD	21-SEP-26	749	Successful
------	-----------	-----	------------

701	501	101	
Nikitha R			
UPI	20-SEP-26	1699	Successful

13. Conclusion

The **Purpller Payment Management System** provides an organized way to manage payment transactions associated with customer orders. The relationship between **Orders and Payment** is maintained using Primary Key and Foreign Key constraints.

SQL operations such as **INSERT, UPDATE, and SELECT** help manage payment information efficiently. Payment reports provide useful information about **successful and failed transactions, payment methods, collected amounts, and customer payment history**, supporting effective payment transaction management.